

PDF 2: Documento de Diseño del Juego (Reglas, Economía y Mecánicas)

1. Algoritmos de Puntuación y Progresión Económica

El sistema de puntuación está diseñado para premiar tanto la velocidad como la precisión ortográfica.

A. Lógica de Scoring (**calculatePhraseScore**)

- **Algoritmo:** La puntuación no es lineal. Se basa en la comparación de arrays de palabras (**target.split(' ')** frente a **input.split(' ')**). Por cada palabra que coincide exactamente en posición y contenido, el sistema otorga **5 puntos**.
- **Acumulación:** La puntuación se actualiza en el **AppState** al finalizar cada frase (evento **Enter**), garantizando que la telemetría refleje el esfuerzo exacto del usuario.
- **Cálculo de Créditos:**
 - **Modo Contrarreloj / Versus:** Los créditos base se calculan mediante **Math.floor(score * 0.1)**. Este valor es escalar y se aplica un multiplicador (**AppState.powerUps.multiplicadorActivo**) que, por defecto, es 1, pero sube a 2 si el usuario tiene equipado un "Contrato VIP".
 - **Modo El Rosco:** Se otorga una recompensa fija de **10 créditos por cada acierto** (letra correcta), totalizando la recompensa según el número de éxitos (**ok * 10**).

2. Mecánicas de Juego y Power-Ups (Gestión de Estado)

Los *Power-Ups* no son solo estéticos; son objetos de inventario con lógica de estado gestionada en **AppState.powerUps**.

A. Gestión de Activos y Pasivos

- **Activación (**activarPowerUp**):** Al activar un activo (ej. **freeze**), el sistema verifica que **AppState.powerUps.inventario[tipo] > 0**. Tras el uso, decrementa el inventario y actualiza el DOM de forma reactiva.
- **Freeze (Tanque de Nitrógeno/Criogénico):**
 - Modifica **AppState.powerUps.tiempoCongelado = true**.
 - Utiliza un **setTimeout** de **5000ms** (5 segundos) para restaurar el estado original del timer, protegiendo al jugador de la cuenta atrás.
- **Rebufo (Mecánica de Combo):**
 - Implementa un contador de racha (**comboAcertadas**). Si el jugador alcanza **3 aciertos consecutivos** (sin errores), el sistema añade automáticamente **+2 segundos** al cronómetro global (**AppState.game.timeLeft**) y dispara una animación de feedback visual.
- **Segunda Oportunidad (Mecánica de Error):**
 - Cuando el jugador comete un error, la función **checkInput** detecta si el usuario tiene **tieneSegundaOportunidad** equipado. Si es así y es la primera

vez que falla (`!chanceUsada`), recorta el valor del input (`val.slice(0, -1)`) y marca el escudo como utilizado, permitiendo al usuario corregir sin penalización.

- **Escudo de Error:** Actúa como un *flag* booleano (`escudoActivo`). Si se activa, el próximo error no cambia el color del borde a `var(--error)` (rojo), sino que lo mantiene neutral o dorado, consumiendo el escudo.

3. Lógica Especial: El Rosco

El Rosco funciona como un autómata de estados independiente (`Rosco` object en `rosco.js`):

- **Generación:** Utiliza un endpoint `/incidencias/game/rosco-ia` que retorna un JSON con 25 objetos `{id: letra, q: definicion, a: respuesta, st: estado}`.
- **Normalización:** Antes de validar la respuesta, el sistema aplica una función `normalize(s)` que convierte el texto a minúsculas, elimina tildes (`normalize("NFD")`) y limpia caracteres especiales. Esto asegura que "canción" sea equivalente a "cancion" para el sistema.
- **Flujo:** El jugador tiene **180 segundos**. Si el tiempo llega a 0 o se recorren las 25 letras, se dispara la función `finish()`, que calcula los créditos y envía la telemetría al endpoint `guardar-partida`.

4. Sistema de Economía y Tienda

- **Validación de Compra:** La función `comprarObjeto` en `shop.js` realiza una validación doble: primero en el cliente (comparando `AppState.currentUser.creditos` vs `precio`) y luego en el servidor, garantizando la integridad de la transacción vía API REST.
- **Equipamiento:** La lógica de equipar objetos modifica tanto el estado local como el persistente. Los cosméticos (temas visuales) tienen un límite de 1 equipado, mientras que las habilidades pasivas permiten hasta 2, gestionado con arrays (`habilidadesEquipadas`) y validaciones de longitud.